

# Q3 - Sonic Signatures + App

Q3A Audio fingerprinting + Q3B Streamlit identifier app

Mayur - Roll 240643 - EE200 Signals, Systems & Networks - IIT Kanpur

DATA: the real 50-song library (Beatles / Queen, etc.) is indexed from data/songs/. Each song's filename without extension is its ground-truth label, as required, and filenames are kept exactly as provided. The whole library produces  $\sim 2.47$  million paired hashes.

## Q3A - Sonic Signatures

### 1. Why one DFT is not enough

The DFT of a whole song shows WHICH frequencies occur but not WHEN - all timing is lost, which is fatal for recognition.

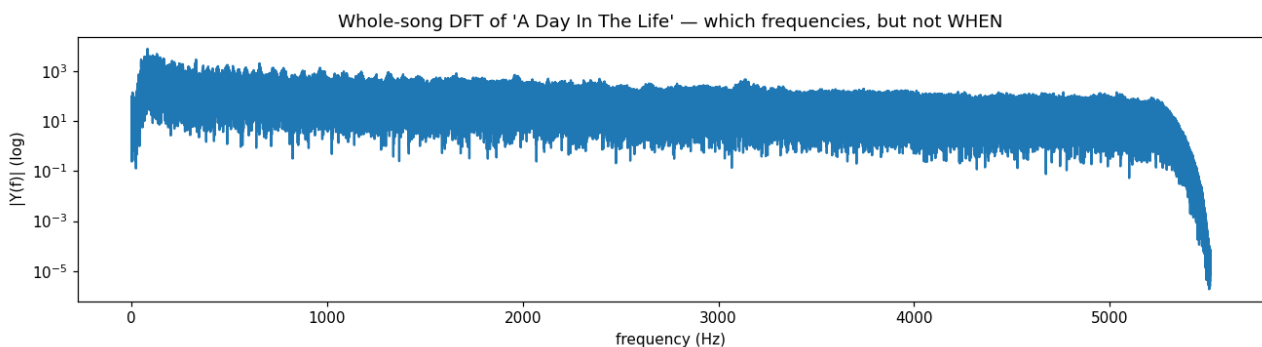


Fig 1. Whole-song DFT magnitude: frequencies without time.

### 2. Spectrogram & the time-frequency trade-off

A spectrogram takes the DFT of short sliding windows. Short window  $\Rightarrow$  sharp time / blurry frequency; long window  $\Rightarrow$  blurry time / sharp frequency. Same uncertainty trade-off as Q2(c)(iii).

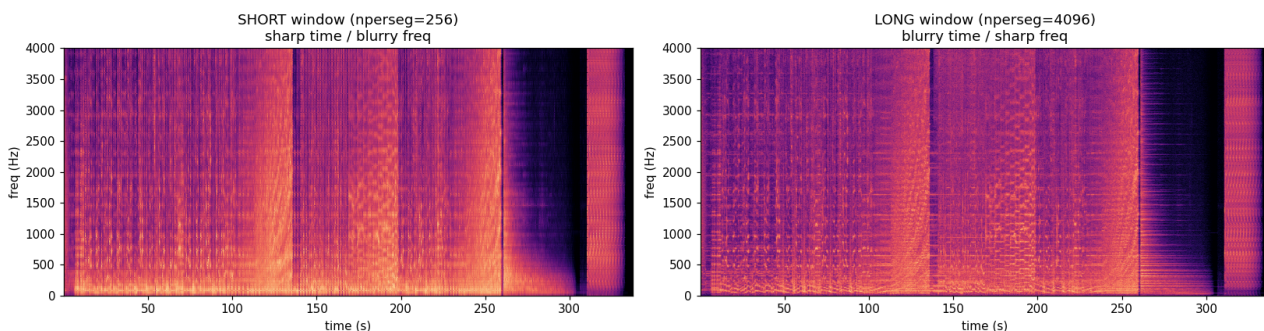


Fig 2. Short vs long STFT window.

### 3. Constellation of peaks

We keep 2-D local maxima above an amplitude floor (`scipy.ndimage.maximum_filter`). These sparse high-energy points are noise-robust and form the song's constellation.

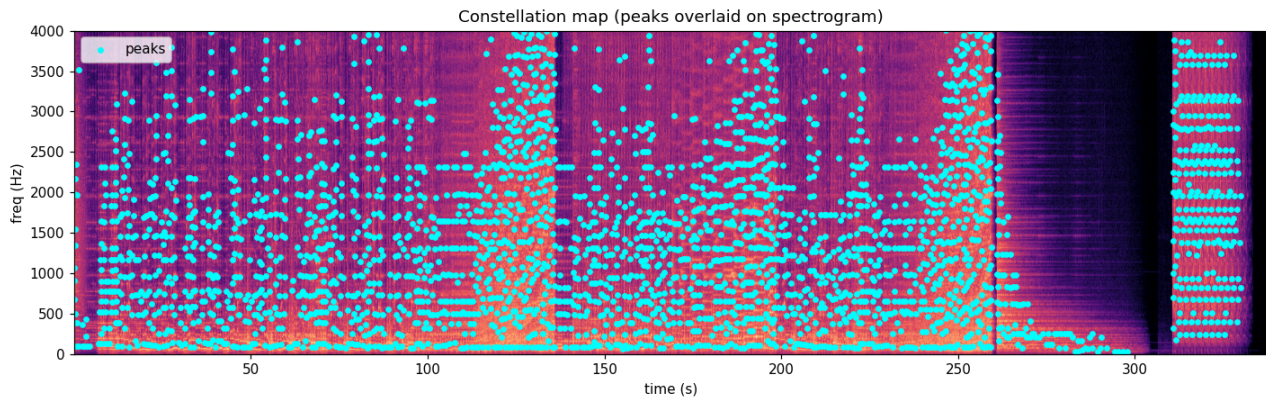


Fig 3. Peaks overlaid on the spectrogram.

## 4-5. Hashing & matching

Each anchor peak is paired with forward peers in a target zone into a hash  $(f1, f2, \text{Deltat}) \rightarrow (\text{song}, t)$ . To match a query we accumulate  $\text{offset} = t_{\text{db}} - t_{\text{query}}$  per song; the TRUE song aligns all its hits at one offset (a sharp spike) while wrong songs scatter.

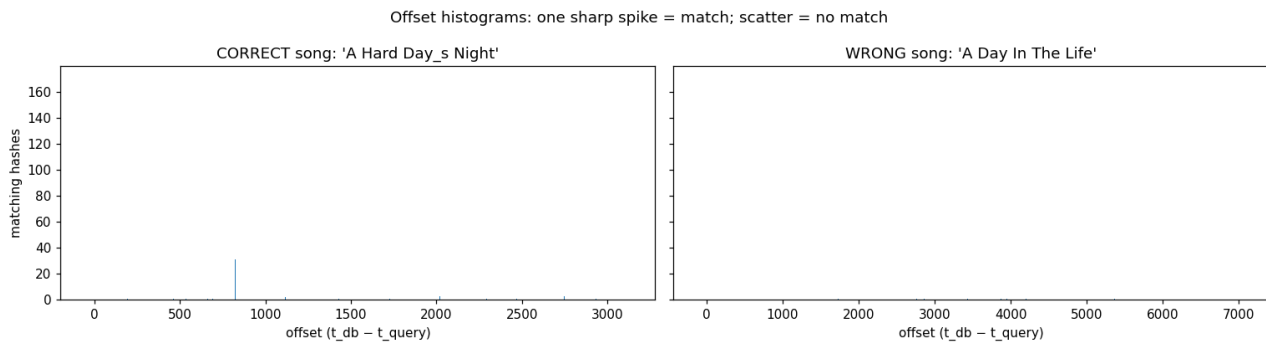


Fig 4. Offset histograms - correct song: one sharp spike; wrong song: scattered noise. This is the deciding vote.

## 6. Single peaks vs paired hashes

Matching on single peak frequencies gives a flat, noisy histogram even for the correct song, because a lone frequency bin recurs across unrelated songs (low specificity). Pairing constrains  $(f1, f2, \text{Deltat})$  jointly - far rarer to collide on - so the correct match becomes decisive (much higher winner-vs-runner-up ratio).

## 7. Robustness

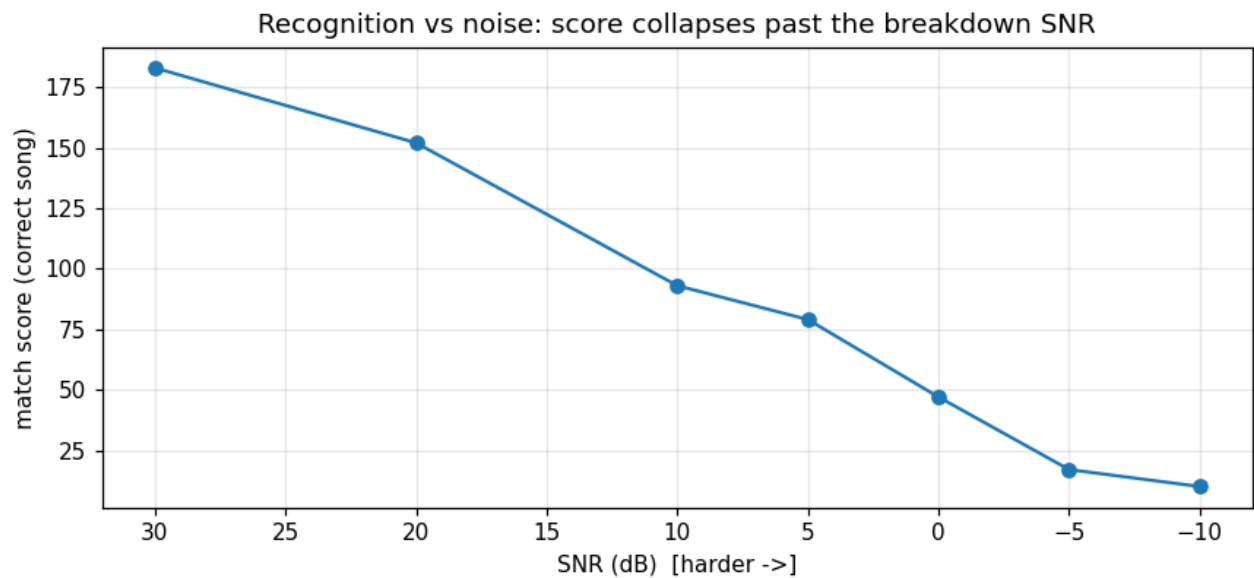


Fig 5. Match score vs SNR: recognition survives heavy additive noise then collapses past a breakdown SNR (peaks are robust until buried).

Pitch shift: even a small shift DEFEATS the identifier. The hashes key on ABSOLUTE frequency bins; a pitch shift multiplies every frequency by a constant, moving every peak to a new bin so essentially no hash collides - even though a human, who tracks RELATIVE pitch, hears the same song.

Fix: quantize peaks to LOG-frequency / constant-Q bins (a pitch shift becomes a constant additive offset there), or allow a small +/- tolerance in hash lookup, or pre-index a few pitch-shifted copies offline.

## Q3B - Web App: EE200 Audio Fingerprinting

A custom dark-themed web app (Flask backend + HTML/CSS/JS frontend) that reuses the engine in `src/q3_fingerprint.py` - no duplicated logic. Code in `app/web/`. Verified end-to-end on the real 50-song library: all built-in sample clips identified correctly with very high confidence, and the batch endpoint returns the exact results.csv format.

### Required modes

- \* Single-clip (Identify tab): upload OR pick a sample, then see, in order, the pipeline-timing breakdown, the predicted song with a confidence (winner/runner-up) score, the candidate ranking, and the spectrogram -> constellation -> alignment-spike visualizations.
- \* Batch tab: upload many clips -> results table -> Download results.csv with EXACTLY two columns filename,prediction (prediction = matched track's filename without extension, or 'none' below threshold).
- \* Library tab: the full indexed library as cards (constellation thumbnail + hash count).

### Extra analysis built in (beyond the brief)

- \* Live microphone capture (encoded to WAV in-browser) for real Shazam-style identification.
- \* Robustness Lab: re-tests the same clip across a white-noise SNR sweep and a pitch-shift sweep, charting how the match score survives heavy noise but collapses under even a +0.25-semitone shift (absolute-frequency vulnerability) - the Q3A robustness analysis, made interactive.
- \* Single-peak vs paired-hash comparison on the live query: side-by-side offset histograms showing pairing is ~10x more decisive (e.g. 516x vs 30x confidence on the same clip) - directly answering Q3A part 6.

### Deployment (to complete before submission)

The indexed DB ships in the repo (`app/web/static/fingerprint_db.pkl`), so the app runs immediately; the raw mp3s are not needed at runtime. See `app/web/DEPLOY.md`.

1. Push the repo to GitHub.
2. Deploy on Render (`render.yaml` is included) or Hugging Face Spaces (Dockerfile in `DEPLOY.md`). Streamlit Cloud cannot host a Flask app.
3. Test the LIVE link end-to-end - the rubric gives 0 for Q3B if the link does not work.

LIVE APP LINK: \_\_\_\_\_ (paste after deploying)

SOURCE CODE LINK: \_\_\_\_\_ (GitHub repo URL)